

Breeze

Breeze è un pacchetto di autenticazione leggero per Laravel che fornisce un'implementazione semplice e pronta all'uso per funzionalità come login, registrazione, recupero password e verifica email. È basato su Blade per le viste e Tailwind CSS per lo stile, rendendolo ideale per applicazioni che non richiedono funzionalità avanzate come quelle offerte da Jetstream.

1. Installazione di Breeze

1. Installa Breeze tramite Composer:

```
composer require laravel/breeze --dev
```

2. Esegui il comando di installazione:

```
php artisan breeze:install
```

3. Installa le dipendenze Node.js e compila gli asset:

```
npm install && npm run dev
```

4. Esegui le migrazioni per creare le tabelle necessarie:

```
php artisan migrate
```

2. Struttura Aggiunta da Breeze

Dopo l'installazione, Breeze aggiunge diverse cartelle e file al tuo progetto:

- **Controller:**

- `Auth/LoginController` : Gestisce il login.
- `Auth/RegisterController` : Gestisce la registrazione.
- `Auth/ForgotPasswordController` : Gestisce il recupero password.
- `Auth/VerifyEmailController` : Gestisce la verifica email.

- **Viste:**
 - `resources/views/auth/` : Contiene le viste per login, registrazione, recupero password, ecc.
 - `resources/views/layouts/` : Contiene il layout principale dell'applicazione.
 - **Rotte:**
 - `routes/auth.php` : Aggiunge le rotte per l'autenticazione.
 - **Middleware:**
 - `EnsureEmailsVerified` : Verifica che l'email dell'utente sia stata confermata.
-

3. Funzionalità di Breeze

Breeze fornisce le seguenti funzionalità:

1. Login:

- Gli utenti possono accedere con email e password.
- La vista si trova in `resources/views/auth/login.blade.php` .

2. Registrazione:

- Gli utenti possono registrarsi con nome, email e password.
- La vista si trova in `resources/views/auth/register.blade.php` .

3. Recupero Password:

- Gli utenti possono richiedere un link per reimpostare la password.
- La vista si trova in `resources/views/auth/forgot-password.blade.php` .

4. Verifica Email:

- Gli utenti devono verificare il loro indirizzo email dopo la registrazione.
 - La vista si trova in `resources/views/auth/verify-email.blade.php` .
-

4. Esempi Pratici

Esempio 1: Personalizzare la Vista di Login

Puoi personalizzare la vista di login modificando il file

`resources/views/auth/login.blade.php` .

```

<!-- resources/views/auth/login.blade.php →
@extends('layouts.app')

@section('content')
<div class="container">
  <h1 class="text-center">Accedi al tuo account</h1>
  <form method="POST" action="{{ route('login') }}">
    @csrf
    <div>
      <label for="email">Email</label>
      <input id="email" type="email" name="email" required autofocus>
    </div>
    <div>
      <label for="password">Password</label>
      <input id="password" type="password" name="password" required
    >
    </div>
    <button type="submit">Accedi</button>
  </form>
</div>
@endsection

```

Esempio 2: Aggiungere un Campo alla Registrazione

Per aggiungere un campo alla registrazione, modifica il file

`resources/views/auth/register.blade.php` e il controller `RegisterController`.

1. Modifica la vista:

```

<!-- resources/views/auth/register.blade.php →
<div>
  <label for="phone">Telefono</label>
  <input id="phone" type="text" name="phone" required>
</div>

```

2. Modifica il controller:

```
// app/Http/Controllers/Auth/RegisterController.php
protected function validator(array $data) {
    return Validator::make($data, [
        'name' => ['required', 'string', 'max:255'],
        'email' => ['required', 'string', 'email', 'max:255', 'unique:users'],
        'password' => ['required', 'string', 'min:8', 'confirmed'],
        'phone' => ['required', 'string', 'max:20'], // Aggiungi questa linea
    ]);
}

protected function create(array $data) {
    return User::create([
        'name' => $data['name'],
        'email' => $data['email'],
        'password' => Hash::make($data['password']),
        'phone' => $data['phone'], // Aggiungi questa linea
    ]);
}
```

3. Aggiorna la migrazione degli utenti:

```
// database/migrations/xxxx_xx_xx_create_users_table.php
public function up() {
    Schema::create('users', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email')->unique();
        $table->string('phone'); // Aggiungi questa linea
        $table->timestamp('email_verified_at')->nullable();
        $table->string('password');
        $table->rememberToken();
        $table->timestamps();
    });
}
```

Esempio 3: Proteggere una Rota con Autenticazione

Puoi proteggere una rota richiedendo che l'utente sia autenticato.

```
// routes/web.php
Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware('auth');
```

Esempio 4: Verifica Email

Breeze gestisce automaticamente la verifica email. Puoi personalizzare il messaggio di verifica modificando il file `resources/views/auth/verify-email.blade.php`.

```
<!-- resources/views/auth/verify-email.blade.php -->
@extends('layouts.app')

@section('content')
<div class="container">
    <h1>Verifica il tuo indirizzo email</h1>
    <p>Ti abbiamo inviato un link di verifica al tuo indirizzo email.</p>
    <form method="POST" action="{{ route('verification.send') }}">
        @csrf
        <button type="submit">Invia di nuovo il link</button>
    </form>
</div>
@endsection
```